

BLOC #01

ESTRUCTURA

```
void setup() {
```

Inici del bloc de configuració (execució única)

BLOC #02

ESTRUCTURA

```
void loop() {
```

Inici del bucle principal del programa

BLOC #03

CLAUDÀTOR

```
}
```

Tancament de bloc / funció

BLOC #04

CLAUDÀTOR

```
}
```

Tancament de bloc / funció (còpia #2)

BLOC #05

CLAUDÀTOR

```
}
```

Tancament de bloc / funció (còpia #3)

BLOC #06

CLAUDÀTOR

```
}
```

Tancament de bloc / funció (còpia #4)

BLOC #07

COMENTARI

```
// _____
```

Comentari explicatiu per al programa

BLOC #08

LLIBRERIA

```
#include <_____.h>
```

Importar llibreria de components

BLOC #09

DIGITALE/S

```
pinMode( PIN , OUTPUT );
```

Configurar pin digital com a SORTIDA d'actuador

BLOC #10

DIGITALE/S

```
pinMode( PIN , INPUT );
```

Configurar pin digital com a ENTRADA de sensor

BLOC #11

DIGITALE/S

```
pinMode( PIN , INPUT_PULLUP );
```

Configurar pin amb resistència interna Pull-Up

BLOC #12

DIGITAL SORTIDA

```
digitalWrite( PIN , HIGH );
```

Activar sortida digital a 5V (Encendre)

BLOC #13

DIGITAL SORTIDA

```
digitalWrite( PIN , HIGH );
```

Activar sortida digital a 5V (Còpia #2)

BLOC #14

DIGITAL SORTIDA

```
digitalWrite( PIN , LOW );
```

Desactivar sortida digital a 0V (Apagar)

BLOC #15

DIGITAL SORTIDA

```
digitalWrite( PIN , LOW );
```

Desactivar sortida digital a 0V (Còpia #2)

BLOC #16

DIGITAL ENTRADA

```
int estat = digitalRead( PIN );
```

Llegir estat digital d'un botó o sensor (HIGH/LOW)

✚ BLOC #17

TEMPS

```
delay( 1000 );
```

i Espera de 1 segon exacte (1000 mil·lisegons)



✚ BLOC #18

TEMPS

```
delay(     );
```

i Pausa personalitzable en mil·lisegons



✚ BLOC #19

TEMPS

```
delay(     );
```

i Pausa personalitzable (Còpia #2)



✚ BLOC #20

TEMPS

```
unsigned long ara = millis();
```

i Llegir rellotge intern en mil·lisegons



✚ BLOC #21

ANALÒGIC

```
int valor = analogRead( A     );
```

i Llegir entrada analògica A0..A5 (0 a 1023)



✚ BLOC #22

ANALÒGIC

```
int llum = analogRead( A0 );
```

i Llegir sensor de llum LDR o potenciómetre



✚ BLOC #23

PWM SORTIDA

```
analogWrite( PIN~ , VALOR );
```

i Sortida PWM proporcional entre 0 i 255



✚ BLOC #24

PWM SORTIDA

```
analogWrite( PIN~ , VALOR );
```

i Sortida PWM proporcional (Còpia #2)



✚ BLOC #25

CONVERSIÓ

```
map( val , 0, 1023, 0,     );
```

i Mapejar rang (ex: de 0..1023 a 0..180 graus)



✚ BLOC #26

VARIABLE

```
int nom = valor;
```

i Declarar o modificar una variable entera



✚ BLOC #27

CONDICIONAL

```
if ( CONDICIÓ ) {
```

i Inici de decisió SI es compleix la condició



✚ BLOC #28

CONDICIONAL

```
} else {
```

i Bloc alternatiu SINÓ del condicional if



✚ BLOC #29

COMPARACIÓ

```
var = HIGH
```

i Peça lògica d'igualtat per posar dins del if



✚ BLOC #30

COMPARACIÓ

```
var < 300
```

i Peça lògica menor que (ex: foscor o obstacle)



✚ BLOC #31

BUCLE FOR

```
for (int i=0; i < 10; i++) {
```

i Bucle per repetir una acció N vegades



✚ BLOC #32

BUCLE WHILE

```
while ( CONDICIÓ ) {
```

i Bucle mentre es compleixi una condició



✚ BLOC #33

MONITOR SÈRIE

```
Serial.begin(9600);
```

i Iniciar comunicació amb el PC al setup()



✚ BLOC #34

MONITOR SÈRIE

```
Serial.println( dada );
```

i Enviar dades o text al Monitor Sèrie amb salt



✚ BLOC #35

ULTRASONS

```
long t = pulseIn(ECHO, HIGH);
```

i Llegir temps de retorn de l'eco d'ultrasons



✚ BLOC #36

ULTRASONS

```
int cm = t / 58;
```

i Convertir temps d'ultrasons en centímetres



✚ BLOC #37

CLIMA DHT

```
float temp = dht.readTemperature();
```

i Llegir temperatura en graus Celsius del DHT11



✚ BLOC #38

CLIMA DHT

```
float hum = dht.readHumidity();
```

i Llegir percentatge d'humitat del DHT11



✚ BLOC #39

SENSOR IR

```
int linia = digitalRead( PIN );
```

i Llegir sensor seguidor de línia o presència PIR



✚ BLOC #40

JOYSTICK

```
int eixX = analogRead(A0);
```

i Llegir posició X del joystick analògic



✚ BLOC #41

SERVOMOTOR

```
brac.attach( PIN );
```

i Enllaçar servomotor al pin digital (al setup)



✚ BLOC #42

SERVOMOTOR

```
brac.write( GRAUS );
```

i Moure servomotor a l'angle indicat (0° a 180°)



✚ BLOC #43

BRUNZIDOR

```
tone( PIN , HZ , MS );
```

i Fer sonar nota musical en Hertz durant ms



✚ BLOC #44

BRUNZIDOR

```
noTone( PIN );
```

i Aturar immediatament el so del brunzidor



✚ BLOC #45

PANTALLA LCD

```
lcd.setCursor( COL , FILA );
```

i Posicionar cursor del LCD I2C (columna 0..15, fila 0..1)



✚ BLOC #46

PANTALLA LCD

```
lcd.print( "_____" );
```

i Mostrar text a la pantalla LCD 16x2



✚ BLOC #47

MOTOR DC L298N

```
digitalWrite(IN1, HIGH); digitalWrite  
(IN2, LOW);
```

i Girar motor DC cap ENDAVANT al controlador



✚ BLOC #48

MOTOR DC L298N

```
analogWrite(ENA, VELOCITAT);
```

i Regular velocitat del motor DC (0 a 255)



BLOC #49

DIGITALE/S

```
pinMode( PIN , OUTPUT );
```

Configurar pin digital com a SORTIDA (Còpia #2)

BLOC #51

DIGITAL SORTIDA

```
digitalWrite( PIN , HIGH );
```

Activar sortida digital a 5V (Còpia #3)

BLOC #53

DIGITAL SORTIDA

```
digitalWrite( PIN , LOW );
```

Desactivar sortida digital a 0V (Còpia #3)

BLOC #55

TEMPS

```
delay( );
```

Pausa personalitzable (Còpia #3)

BLOC #50

DIGITALE/S

```
pinMode( PIN , OUTPUT );
```

Configurar pin digital com a SORTIDA (Còpia #3)

BLOC #52

DIGITAL SORTIDA

```
digitalWrite( PIN , HIGH );
```

Activar sortida digital a 5V (Còpia #4)

BLOC #54

DIGITAL SORTIDA

```
digitalWrite( PIN , LOW );
```

Desactivar sortida digital a 0V (Còpia #4)

BLOC #56

TEMPS

```
delay( );
```

Pausa personalitzable (Còpia #4)

BLOC #57

CLAUDÀTOR

```
}
```

Tancament de bloc / funció (Còpia #5)

BLOC #58

CLAUDÀTOR

```
}
```

Tancament de bloc / funció (Còpia #6)

BLOC #59

CONDICIONAL

```
if ( CONDICIÓ ) {
```

Inici de decisió SI es compleix la condició (Còpia #2)

BLOC #60

CONDICIONAL

```
} else {
```

Bloc alternatiu SINÓ (Còpia #2)

BLOC #61

VARIABLE

```
int nom = valor;
```

Declarar o modificar variable entera (Còpia #2)

BLOC #62

DIGITAL ENTRADA

```
int estat = digitalRead( PIN );
```

Llegir estat digital d'un botó o sensor (Còpia #2)

BLOC #63

ANALÒGIC

```
int valor = analogRead( A_ );
```

Llegir entrada analògica A0..A5 (Còpia #2)

BLOC #64

COMENTARI

```
//
```

Comentari explicatiu per al programa (Còpia #3)